HUMANLY · SPRINT 4

# JOVI AI LensLab

Laboratório fotográfico que mede a foto, explica o que pode melhorar, verifica a nova captura e aprende com cada usuário.

DOCUMENTAÇÃO TÉCNICA

| | |
|---|---|
| DISCIPLINA | Computational Thinking with Python |
| ENTREGA | Sprint 4 — implementação final |
| TURMA | 1ESPI |
| INTEGRANTES | Eduardo Bechara Medeiros Craveiro RM 571081<br>Gustavo Moita de Lima RM 569180<br>Bruno Carreiro Dos Santos RM 569423 |
| DATA | 23 de outubro de 2026 |

# Sumário

O documento segue a ordem dos critérios de avaliação: descrição do projeto, funcionalidades, requisitos técnicos da Sprint 4, alterações em relação à Sprint 3 e evidências de testes.

1. Visão geral do projeto . . . 3

2. Objetivos e justificativa . . . 4

3. Funcionalidades implementadas . . . 4

4. Arquitetura da solução . . . 6

5. Persistência em arquivos JSON . . . 7

6. Integração com o restante da solução . . . 9

7. Consumo de API externa . . . 12

8. Validação de entradas e tratamento de exceções . . . 13

9. Aprendizado individualizado . . . 14

10. Alterações em relação à Sprint 3 . . . 15

11. Atendimento aos critérios e testes . . . 16

12. Como executar e demonstrar . . . 17

13. Limitações, referências e conclusão . . . 18

SEÇÃO 01

# Visão geral do projeto

## Problema

Os smartphones da linha JOVI oferecem modos avançados de câmera, como Noturno, Retrato e HDR, mas o usuário raramente sabe quando cada recurso ajuda, que efeito colateral ele traz e se a mudança realmente melhorou a foto. Recomendações fixas, como as da Sprint 2, ignoram a foto real, a intenção de quem fotografa e o histórico de cada pessoa.

## Proposta

O **JOVI AI LensLab** é um programa de terminal em Python que trata a fotografia como um experimento. Ele mede a foto, interpreta a cena, simula alternativas, pede uma nova captura, verifica o resultado e ajusta as próximas recomendações para aquele usuário. O programa não edita a imagem: ele ensina o usuário a fotografar melhor.




*Figura 1 — Ciclo de uso. Cada etapa produz um tipo de informação identificado na interface.*

## Como ler os rótulos da interface

A interface sempre separa o que foi calculado, o que foi inferido e o que foi comprovado. Essa separação evita apresentar uma estimativa como se fosse um fato.

| Rótulo | Origem | Significado |
|---|---|---|
| MEDIDO | Cálculo local | Valores obtidos dos pixels: brilho, contraste, nitidez, saturação, recortes e resolução. |
| IA | API Gemini | Interpretação da cena (assunto, composição, movimento), aceita só depois de validada. |
| PREVISTO | Motor de cenários | Ganho estimado antes de o usuário agir; é uma hipótese. |
| VERIFICADO | Recaptura | Diferença real entre a foto original e a nova foto, no mesmo contexto. |

*Tabela 1 — Rótulos usados nos diagnósticos.*

SEÇÃO 02

# Objetivos e justificativa

## Objetivo geral

Desenvolver em Python uma ferramenta que combine análise técnica local, interpretação por IA e verificação por recaptura para produzir recomendações fotográficas explicáveis e personalizadas, com dados persistidos em arquivos JSON e integração com o restante da solução Humanly.

## Objetivos específicos

- Medir indicadores técnicos da foto sem depender de internet.
- Interpretar a cena com a API Gemini apenas quando o usuário consentir.
- Apresentar até três alternativas com ganho previsto, esforço, risco, confiança e evidências.
- Comparar a foto original com uma recaptura no mesmo objetivo e intenção.
- Ajustar as recomendações de cada usuário com base nos resultados verificados.
- Persistir perfis, histórico e aprendizado em arquivos JSON, um por perfil.
- Trocar arquivos JSON com o app da câmera JOVI, que representa o restante da solução.
- Continuar funcionando, com mensagens claras, quando a API ou os arquivos falharem.

## Justificativa

Dizer apenas qual modo usar não ensina. Ao mostrar o motivo de cada sugestão, o risco envolvido e o resultado medido depois, o usuário entende o efeito de cada recurso e passa a escolher sozinho. Para a JOVI, isso aumenta o uso consciente dos recursos avançados da câmera. A verificação por recaptura também protege contra o principal limite da IA: uma interpretação errada nunca é tratada como prova de melhoria.

SEÇÃO 03

# Funcionalidades implementadas

O programa abre com a seleção de perfil e, em seguida, mostra o menu principal. Toda opção volta ao menu ao terminar, e a opção 0 encerra a execução. A tela abaixo foi capturada do próprio programa durante a geração deste documento.

```
• • •   python main.py

======================================================================
                    HUMANLY JOVI AI LENSLAB
======================================================================
Perfil: Ana | Nível: Iniciante | Gemini: chave configurada
  1. Trocar ou criar perfil
  2. Analisar fotografia
  3. Explorar simulação 'E se?'
  4. Comparar antes e depois e aprender
  5. Marcar recomendação para testar
  6. Ver histórico
  7. Ver evolução
  8. Receber orientação sem foto
  9. Privacidade e dados
 10. Integração com o app JOVI (arquivos JSON)
  0. Encerrar
Escolha: 2
```

*Figura 2 — Menu principal real do programa.*

| Opção | Funcionalidade | O que faz |
|---|---|---|
| 1 | Trocar ou criar perfil | Seleciona, cria ou exclui perfis; o primeiro perfil recebe um tour guiado com foto de exemplo. |
| 2 | Analisar fotografia | Abre o explorador de arquivos, mede a foto, consulta a IA se autorizado e mostra um resumo em linguagem simples. |
| 3 | Explorar simulação "E se?" | Gera uma prévia visual da recomendação sem alterar a foto original. |
| 4 | Comparar antes e depois | Analisa a recaptura no mesmo contexto, registra a nota do usuário e atualiza o aprendizado. |
| 5 | Marcar recomendação | Guarda a dica que será testada; ela é sugerida automaticamente na opção 4. |
| 6 | Ver histórico | Lista as análises do perfil ativo. |
| 7 | Ver evolução | Mostra ganho médio, recapturas bem-sucedidas e os conselhos que mais funcionaram. |
| 8 | Orientação sem foto | Recomenda com base no ambiente descrito pelo usuário (herança da Sprint 2). |
| 9 | Privacidade e dados | Altera o consentimento, exporta um resumo ou exclui o perfil e seus arquivos. |
| 10 | Integração com o app JOVI | Processa solicitações JSON recebidas e exporta o perfil para o app. |
| 0 | Encerrar | Finaliza a execução; Ctrl+C também encerra sem mensagem de erro técnico. |

*Tabela 2 — Funcionalidades do menu principal.*

## Exemplo de resultado para o usuário

O diagnóstico aparece primeiro em linguagem simples; os detalhes técnicos só são exibidos se o usuário pedir. Saída real para a foto `samples/exemplo_escuro.jpg`:

```
• • •   opção 2 — analisar fotografia

============================================================
RESULTADO DA SUA FOTO
============================================================
Nota geral: 37/100  [#######-------------]  - Precisa de ajustes importantes

O que está bom:
  + Cores naturais e equilibradas

O que pode melhorar:
  - Está muito escura ou muito clara
  - Está desfocada
  - Resolução baixa, pouco espaço pra recortar

Nossa principal dica: Estabilizar Câmera
  Apoiar o aparelho ou usar temporizador reduz o tremor provocado pelas mãos.
  (melhoria grande esperada)
  (+ 2 outra(s) sugestão(ões) nos detalhes completos)

Obs: a leitura extra por IA não estava disponível agora; usamos só as medidas da foto.
```

*Figura 3 — Resumo da análise exibido ao usuário.*

SEÇÃO 04

# Arquitetura da solução

O código é dividido em camadas. Cada camada só conversa com a de baixo, e as regras de negócio ficam em funções que recebem parâmetros e devolvem valores, sem ler o teclado nem imprimir na tela. Isso permite testar as regras isoladamente e trocar a forma de armazenamento sem mexer no resto.

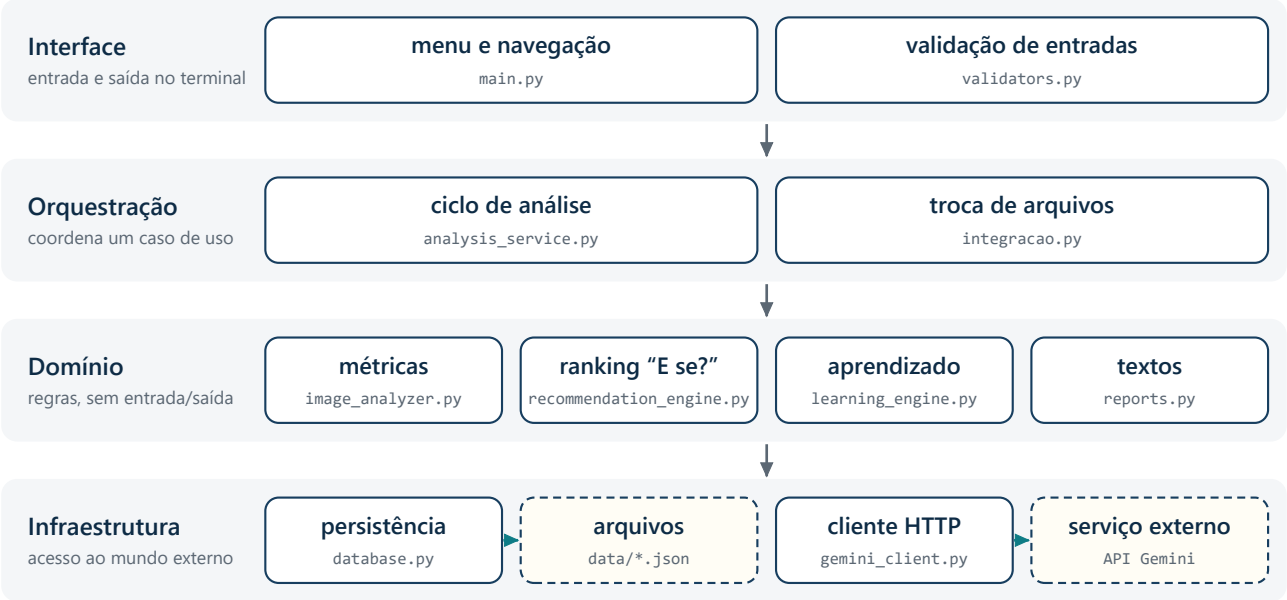



*Figura 4 — Camadas e módulos. Caixas tracejadas representam recursos fora do código.*

| Módulo | Responsabilidade | Exemplo de função (assinatura) |
|---|---|---|
| main.py | Menu, perguntas e navegação | `menu_integracao(usuario, pasta_dados, base) -> None` |
| validators.py | Validação de entradas do usuário | `validar_inteiro(valor, minimo, maximo, nome) -> int` |
| analysis_service.py | Ciclo de análise completo | `analisar_fotografia(usuario, caminho_imagem, ...) -> dict` |
| integracao.py | Contrato de troca de arquivos | `validar_solicitacao(dados) -> dict` |
| image_analyzer.py | Métricas e cópia segura para a API | `analisar_imagem(caminho) -> dict` |
| recommendation_engine.py | Pontuação e cenários "E se?" | `simular_intervencoes(problemas, movimento, ...) -> list[dict]` |
| learning_engine.py | Comparação e ajuste de pesos | `calcular_ajuste_peso(ganho_real, avaliacao) -> float` |
| database.py | Persistência em arquivos JSON | `salvar_analise_com_recomendacoes(analise, ...) -> int` |
| gemini_client.py | Cliente da API externa | `analisar_com_gemini(caminho_imagem, objetivo, intencao) -> dict` |
| reports.py | Textos e relatórios | `formatar_resumo_analise(resultado) -> str` |

| Módulo | Responsabilidade | Exemplo de função (assinatura) |
|---|---|---|
| textos.py | Rótulos acentuados para códigos internos | `legivel(codigo) -> str` |

*Tabela 3 — Módulos e uma função representativa de cada um.*

**DECISÃO TÉCNICA**

A IA é injetada como parâmetro (`cliente_ia`) em `analisar_fotografia`. Com isso, os testes usam um cliente falso, sem rede, e o modo de demonstração usa respostas simuladas pela mesma interface.

SEÇÃO 05

# Persistência em arquivos JSON

Todos os dados ficam em arquivos JSON legíveis, criados automaticamente. Há um índice com os perfis e um arquivo para cada perfil, o que deixa o isolamento entre usuários visível na própria estrutura de pastas: excluir um perfil apaga apenas o arquivo dele.

estrutura de pastas

```
data/
├── usuarios.json           perfis, consentimento e contadores de identificadores
└── perfis/
    ├── perfil_1.json       análises, recomendações, comparações e pesos do perfil 1
    └── perfil_2.json       ... de outro usuário, sem mistura de dados
```

## Conteúdo de um arquivo de perfil

Trecho real gerado pelo programa após uma análise (campos repetitivos omitidos):

data/perfis/perfil_1.json


```
{
  "versao": 1,
  "usuario_id": 1,
  "analises": [
    {
      "id": 1,
      "nome_arquivo": "exemplo_escuro.jpg",
      "objetivo": "retrato",
      "modo_execucao": "offline",
      "pontuacao_geral": 36.87,
      "metricas": {
        "brilho": 9.62,
        "nitidez_bruta": 19.88,
        "contexto_chave": "retrato|escuro|estatico|iniciante|natural"
      },
      "...": "demais campos omitidos neste documento"
    }
  ],
  "recomendacoes": [
    {
      "id": 1,
      "analise_id": 1,
      "codigo_acao": "estabilizar_camera",
      "ganho_previsto": 35.0,
      "marcado_em": null
    }
  ],
  "comparacoes": [],
  "pesos_aprendizado": []
}
```


## Gravação segura

Se o programa fosse interrompido no meio de uma gravação, o arquivo poderia ficar pela metade. Por isso o conteúdo é escrito primeiro em um arquivo temporário e só depois substitui o original com `os.replace`, que troca o arquivo de uma vez só.

database.py — _gravar_json

```python
def _gravar_json(arquivo: Path, dados: dict) -> None:
    arquivo.parent.mkdir(parents=True, exist_ok=True)
    temporario = arquivo.with_name(arquivo.name + ".tmp")
    try:
        with open(temporario, "w", encoding="utf-8") as stream:
            json.dump(dados, stream, ensure_ascii=False, indent=2)
        os.replace(temporario, arquivo)
    except OSError as exc:
        if temporario.exists():
            temporario.unlink()
        raise ArmazenamentoError(f"Não foi possível gravar {arquivo.name}: {exc.strerror}.") from exc
```

## Leitura com tratamento de erros

Na leitura, um arquivo inexistente vira uma estrutura vazia, e um arquivo corrompido gera uma mensagem que indica qual arquivo restaurar, em vez de encerrar o programa com erro técnico.

database.py — _ler_json

```python
def _ler_json(arquivo: Path, padrao: dict) -> dict:
    if not arquivo.exists():
        return copy.deepcopy(padrao)
    try:
        with open(arquivo, encoding="utf-8") as stream:
            dados = json.load(stream)
    except json.JSONDecodeError as exc:
        raise ArmazenamentoError(
            f"O arquivo de dados {arquivo.name} está corrompido (linha {exc.lineno}). "
            "Restaure uma cópia ou apague-o para recomeçar."
        ) from exc
    except OSError as exc:
        raise ArmazenamentoError(f"Não foi possível ler {arquivo.name}: {exc.strerror}.") from exc
    if not isinstance(dados, dict):
        raise ArmazenamentoError(f"O arquivo {arquivo.name} não tem o formato esperado.")
    for chave, valor in padrao.items():
        dados.setdefault(chave, copy.deepcopy(valor))
    return dados
```

## Regras que antes eram do banco de dados

| Regra | Onde é garantida |
|---|---|
| Apelido único (sem diferenciar maiúsculas) | `database.criar_usuario` → RegistroDuplicadoError |
| Nível e estilo dentro das opções válidas | `database.criar_usuario` |
| Pontuação entre 0 e 100 | `database.salvar_analise_com_recomendacoes` |
| Avaliação de 1 a 5 e peso entre 0,50 e 1,50 | `database.registrar_comparacao` |
| Mesma comparação não é registrada duas vezes | `database.registrar_comparacao` → RegistroDuplicadoError |
| Identificadores únicos entre perfis | contadores em `usuarios.json` |

*Tabela 4 — Validações que substituem as restrições do SQLite da Sprint 3.*

SEÇÃO 06

# Integração com o restante da solução

A Sprint 4 pede integração por arquivos ou API. O LensLab usa **troca de arquivos JSON** com o app da câmera JOVI: o app deixa um pedido numa pasta e o LensLab devolve o resultado em outra. Os dois lados só precisam combinar o formato dos arquivos, documentado abaixo.

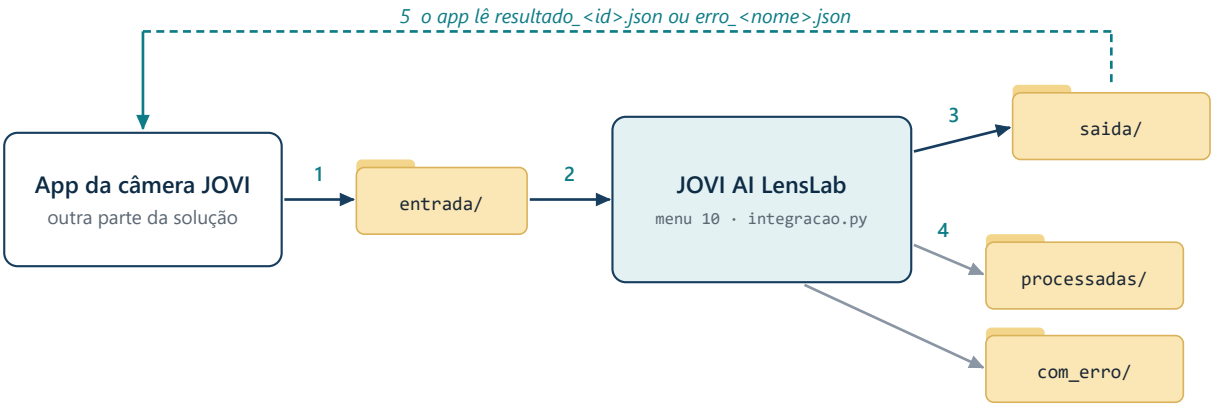


*Figura 5 — Fluxo da integração: (1) o app grava o pedido, (2) o LensLab lê, (3) grava o resultado, (4) arquiva o pedido e (5) o app lê a resposta.*

## Contrato da solicitação (versão 1.0)

| Campo | Tipo | Regra de validação |
|---|---|---|
| `versao` | texto | Precisa ser da versão 1.x do contrato. |
| `id_solicitacao` | texto | 1 a 60 caracteres: letras, números, hífen ou sublinhado. Dá nome ao resultado. |
| `perfil` | objeto | apelido (2 a 40 caracteres), nível, estilo e consentimento (true/false). |
| `foto` | texto | Caminho absoluto ou relativo à raiz do projeto; o arquivo precisa existir e ser imagem válida. |
| `objetivo` | texto | documento, pessoa, retrato, paisagem, produto, redes_sociais ou outro. |
| `intencao` | texto | 3 a 160 caracteres descrevendo o que o usuário queria com a foto. |

*Tabela 5 — Campos da solicitação enviada pelo app.*

`integracao/exemplos/solicitacao_exemplo.json`

```
{
  "versao": "1.0",
  "origem": "app_camera_jovi",
  "id_solicitacao": "exemplo-001",
  "perfil": {
    "apelido": "Demo App JOVI",
    "nivel": "iniciante",
    "estilo": "natural",
    "consentimento_api": false
  },
  "foto": "samples/exemplo_escuro.jpg",
  "objetivo": "retrato",
  "intencao": "foto de perfil para rede social"
}
```

## Resultado devolvido ao app

Trecho real do arquivo gerado para a solicitação acima. O resultado não inclui o hash da foto, metadados nem localização.

integracao/saida/resultado_exemplo-001.json


```
{
  "versao": "1.0",
  "tipo": "resultado_analise",
  "status": "concluido",
  "id_solicitacao": "exemplo-001",
  "processado_em": "2026-10-23T14:30:00+00:00",
  "perfil": {
    "id": 2,
    "apelido": "Demo App JOVI",
    "criado_nesta_solicitacao": true
  },
  "analise": {
    "id": 2,
    "modo_execucao": "offline",
    "pontuacao_geral": 36.87,
    "classificacao": "Precisa de ajustes importantes",
    "dimensoes": {
      "exposicao": 4.69,
      "nitidez": 17.83,
      "contraste": 55.18,
      "...": "omitidas"
    },
    "pontos_fortes": [
      "cores naturais e equilibradas"
    ],
    "pontos_a_melhorar": [
      "está muito escura ou muito clara",
      "está desfocada",
      "resolução baixa, pouco espaço pra recortar"
    ]
  },
  "ia": {
    "disponivel": false,
    "motivo_indisponivel": "consentimento ou chave não configurados",
    "assunto_principal": null,
    "tipo_cena": null
  },
  "recomendacoes": [
    {
      "posicao": 1,
      "codigo_acao": "estabilizar_camera",
      "titulo": "Estabilizar Câmera",
      "ganho_previsto": 35,
      "confianca": "media",
      "esforco": "baixo",
      "explicacao": "Apoiar o aparelho ou usar temporizador reduz o tremor provocado pelas mãos.",
    ...
```


## Tratamento de falhas

| Situação | O que acontece |
|---|---|
| JSON malformado ou com acento salvo fora de UTF-8 | Pedido vai para `com_erro/` e `saida/erro_<nome>.json` explica a linha e a coluna. |
| Campo ausente ou fora da regra | Mesmo tratamento, com a mensagem da regra violada. |
| Foto inexistente ou corrompida | Mesmo tratamento; nenhuma análise é gravada. |
| Perfil já existente | Perfil é reaproveitado; o consentimento dele não pode ser alterado pelo app. |
| Arquivo de dados local corrompido | O lote é interrompido e o pedido permanece na entrada para ser reprocessado. |

*Tabela 6 — Comportamento da integração em cada falha prevista.*

integracao.py — processar_arquivo

```python
def processar_arquivo(
    arquivo: Path,
    pasta_dados: str | Path = DADOS_DIR,
    base: str | Path = INTEGRACAO_DIR,
    cliente_ia: ClienteIA | None = None,
) -> dict:
    """Atende uma solicitacao; nunca deixa o arquivo parado na entrada."""
    pastas = preparar_pastas(base)
    try:
        solicitacao = validar_solicitacao(ler_solicitacao(arquivo))
        usuario, criado = obter_ou_criar_perfil(solicitacao["perfil"], pasta_dados)
        analise = analisar_fotografia(
            usuario, solicitacao["foto"], solicitacao["objetivo"], solicitacao["intencao"],
            usuario["estilo_preferido"], bool(usuario["consentimento_api"]), pasta_dados,
            cliente_ia or cliente_ia_padrao(),
        )
    except database.ArmazenamentoError:
        raise  # problema nos dados locais, nao na solicitacao: interrompe o lote
    except (ValueError, OSError) as exc:
        mensagem = str(exc) or type(exc).__name__
        saida = _gravar_saida(pastas["saida"] / f"erro_{arquivo.stem}.json", {
            "versao": VERSAO_CONTRATO_INTEGRACAO, "tipo": "erro_solicitacao", "status": "erro",
            "arquivo": arquivo.name, "mensagem": mensagem, "processado_em": database.agora_iso(),
        })
        _arquivar(arquivo, pastas["com_erro"])
        return {"arquivo": arquivo.name, "status": "erro", "mensagem": mensagem, "saida": saida}

    resultado = montar_resultado(solicitacao, usuario, criado, analise)
    saida = _gravar_saida(pastas["saida"] / f"resultado_{solicitacao['id_solicitacao']}.json", resultado)
    _arquivar(arquivo, pastas["processadas"])
    return {
        "arquivo": arquivo.name, "status": "concluido", "saida": saida,
        "mensagem": f"{usuario['apelido']}: nota {analise['pontuacao_geral']:.0f}/100",
    }
```

SEÇÃO 07

# Consumo de API externa

A API externa consumida é a **Google Gemini**, acessada pelo SDK oficial `google-genai`. Ela interpreta o conteúdo da foto, o que as métricas de pixels não conseguem: qual é o assunto, se há movimento e como está a composição.

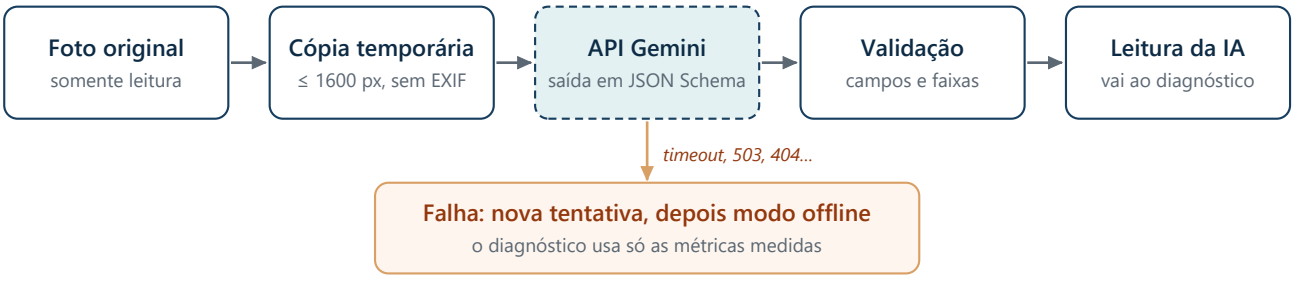



*Figura 6 — Caminho de uma foto até a API e o tratamento de falhas.*

| Aspecto | Implementação |
|---|---|
| Quando é chamada | Somente com consentimento no perfil e chave em GEMINI_API_KEY (variável de ambiente ou .env). |

| Aspecto | Implementação |
|---|---|
| Modelo | `gemini-flash-latest`, configurável em GEMINI_MODEL. |
| Formato da resposta | JSON obrigatório, descrito por JSON Schema; campos e códigos fora da lista são descartados. |
| Tempo limite e repetição | 30 s por chamada; uma nova tentativa com espera em falhas temporárias (timeout, 503, 504). |
| Falhas definitivas | Chave inválida, cota esgotada ou modelo indisponível: nenhuma nova tentativa; segue offline. |
| Privacidade | Envia uma cópia reduzida e sem EXIF/GPS, apagada ao final; a original nunca é alterada. |
| Sem chave | O programa funciona offline. JOVI_DEMO_MODE=1 simula a resposta para apresentações. |

*Tabela 7 — Como a API é consumida.*

SEÇÃO 08

# Validação de entradas e tratamento de exceções

Toda entrada do usuário passa por uma função de validação. Quando a resposta é inválida, a mesma pergunta é repetida com uma explicação, sem recursão e sem encerrar o programa. O valor 0 cancela fluxos com várias perguntas.

validators.py — solicitar_validado

```python
def solicitar_validado(
    mensagem: str,
    validador: Callable[[str], object],
    permitir_cancelar: bool = False,
    padrao: object | None = None,
) -> object | None:
    """Repete uma entrada invalida sem recursao; `0` cancela quando permitido.

    Quando ``padrao`` e informado, Enter em branco reaproveita esse valor
    em vez de exigir nova digitacao.
    """
    while True:
        valor = input(mensagem).strip()
        if permitir_cancelar and valor == "0":
            return None
        if padrao is not None and valor == "":
            return padrao
        try:
            return validador(valor)
        except ValidationError as exc:
            print(f"Entrada inválida: {exc}")
```

| Onde | O que é validado ou tratado | Exceções capturadas |
|---|---|---|
| validators.py | Números em faixa, sim/não, tamanho de textos, caminho, formato e tamanho da imagem | ValidationError, ValueError, OSError |
| image_analyzer.py | Imagem corrompida, renomeada ou abaixo de 320 × 240 | UnidentifiedImageError, OSError |
| gemini_client.py | Resposta da API fora do schema; erros de rede, cota e autenticação | GeminiResponseError, JSONDecodeError, Exception do SDK |

| Onde | O que é validado ou tratado | Exceções capturadas |
|---|---|---|
| database.py | Arquivo de dados ilegível ou corrompido; falha ao gravar | JSONDecodeError, OSError → ArmazenamentoError |
| integracao.py | Pedido malformado, codificação errada, campos inválidos | JSONDecodeError, UnicodeDecodeError, ValueError |
| main.py | Falhas durante a sessão, Ctrl+C e fim da entrada | ValueError, OSError, KeyboardInterrupt, EOFError |

*Tabela 8 — Pontos de validação e tratamento de exceções.*

SEÇÃO 09

# Aprendizado individualizado

Depois de cada recaptura, o peso da recomendação usada é ajustado apenas para aquele usuário e para aquele contexto. O contexto é uma chave formada por cinco partes:

```
chave de contexto
```

```
objetivo | faixa de luz | movimento | nível | estilo
retrato  | escuro      | estatico  | iniciante | natural
```

Assim, uma dica que funcionou num retrato escuro não altera as sugestões para um documento bem iluminado, nem as de outro usuário.

| Ganho real | Nota do usuário (1 a 5) | Leitura dos sinais | Ajuste no peso |
|---|---|---|---|
| ≥ +5 pontos | ≥ 3 | concordam: funcionou | +0,10 |
| abaixo de −5 pontos | ≤ 3 | concordam: não funcionou | −0,10 |
| abaixo de +5 pontos | ≤ 2 | concordam: não funcionou | −0,10 |
| ≥ +5 pontos | ≤ 2 | conflitam | 0,00 |
| abaixo de −5 pontos | ≥ 4 | conflitam | −0,03 |
| entre −5 e +5 pontos | ≥ 4 | preferência pessoal | +0,03 |
| demais combinações | — | inconclusivo | 0,00 |

*Tabela 9 — Regra de ajuste do peso aprendido (learning_engine.calcular_ajuste_peso).*

O peso fica limitado ao intervalo de 0,50 a 1,50 e cada novo ajuste no mesmo contexto é multiplicado por um fator que diminui com o número de comparações já feitas (mínimo de 0,30). Isso impede que poucos exemplos dominem as recomendações.

SEÇÃO 10

# Alterações em relação à Sprint 3

A tabela resume o que foi removido, modificado e adicionado, com a justificativa de cada mudança. O ciclo principal do sistema e as regras de recomendação continuam iguais; a Sprint 4 muda onde os dados ficam e acrescenta a ligação com o app da câmera.

| Tipo | Sprint 3 | Sprint 4 | Justificativa |
|---|---|---|---|
| Removido | Banco SQLite (data/jovi.db) com tabelas e SQL | Sem banco relacional | A Sprint 4 exige persistência em arquivos de texto ou JSON. |
| Modificado | Persistência em SQLite | usuarios.json + um arquivo JSON por perfil | Dados legíveis e isolamento por usuário visível nos arquivos. |
| Modificado | Integridade garantida por transações SQL | Gravação atômica e validações em database.py | Mesma proteção contra dados pela metade, sem banco. |
| Modificado | SQL escrito dentro do motor de aprendizado | Funções de persistência com parâmetros e retorno | Separação de responsabilidades e testes mais simples. |
| Modificado | Menu com 9 opções | Menu com 10 opções | Nova opção de integração. |
| Modificado | Textos da interface sem acentos e códigos crus (ex.: “Estabilizar Camera”) | Textos acentuados e rótulos legíveis gerados por textos.legivel | Interface na língua do estudante (problema P05 da avaliação de Nielsen). |
| Adicionado | — | Integração com o app JOVI por arquivos JSON | Requisito de integração com o restante da solução. |
| Adicionado | — | Contrato versionado com exemplo em integracao/exemplos/ | Outra parte da solução consegue gerar pedidos válidos. |
| Adicionado | — | Exportação do perfil para o app | O app exibe evolução e conselhos aprendidos. |
| Adicionado | — | Mensagem clara para arquivo de dados corrompido | Falha previsível tratada com try/except. |
| Adicionado | — | Sessão segura ao excluir o perfil em uso; Ctrl+C sem traceback | Robustez da interface. |
| Mantido | Ciclo medir, interpretar, simular, recapturar e aprender | Inalterado | É o núcleo da proposta. |
| Mantido | API Gemini com schema e modo offline | Inalterado | Requisito de API externa já atendido. |

*Tabela 10 — Alterações da Sprint 3 para a Sprint 4.*

SEÇÃO 11

# Atendimento aos critérios e testes

| Critério da Sprint 4 | Evidência | Situação |
|---|---|---|
| Menu, retorno ao menu e opção de encerrar | Seção 3 · `main.executar_sessao` | Atendido |
| Funcionalidades de acordo com a proposta | Seções 1 e 3 | Atendido |
| Subrotinas com parâmetros e retorno | Seção 4 · tipos verificados por teste | Atendido |
| Validação de entradas e try/except | Seção 8 | Atendido |
| Persistência em arquivos JSON | Seção 5 | Atendido |
| Consumo de API externa | Seção 7 | Atendido |
| Integração com o restante da solução | Seção 6 | Atendido |
| Boas práticas de programação | Seção 4 · testes automatizados | Atendido |
| Interface intuitiva e mensagens claras | Seção 3 | Atendido |

*Tabela 11 — Mapa entre os critérios de avaliação e o projeto.*

## Testes automatizados

A suíte tem **84 testes** escritos com a biblioteca padrão `unittest`. Eles usam pastas temporárias e um cliente de IA falso, então não dependem de internet, chave de API nem dos dados do usuário. O arquivo `test_requisitos_sprint4.py` tem um grupo de testes para cada critério da tabela acima.

| Arquivo | Testes | O que cobre |
|---|---|---|
| `test_analysis_service.py` | 2 | Modo offline e modo híbrido com cliente de IA controlado |
| `test_armazenamento_json.py` | 9 | Arquivos JSON, gravação atômica, arquivo corrompido, exclusão, duplicidades |
| `test_database_and_learning.py` | 11 | Isolamento por usuário, comparação, pesos e recomendação pendente |
| `test_integracao.py` | 8 | Contrato de integração, pedidos inválidos e exportação de perfil |
| `test_main_sessao.py` | 5 | Exclusão do perfil em uso, Ctrl+C e fim da entrada |
| `test_recommendation_and_gemini.py` | 12 | Pontuação, cenários, validação da resposta da IA e repetição em falhas |
| `test_reports.py` | 7 | Textos em linguagem simples, sem códigos internos |
| `test_requisitos_sprint4.py` | 22 | Um grupo de testes por critério da Sprint 4 |
| `test_validators_and_images.py` | 8 | Validações, métricas, EXIF, cópia temporária e preservação do original |
| Total | **84** | Todos passando na versão entregue |

*Tabela 12 — Distribuição dos testes automatizados.*

SEÇÃO 12

# Como executar e demonstrar

## Instalação e execução

PowerShell (Windows, Python 3.11 ou superior)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python main.py
```

## API Gemini (opcional)

Crie um arquivo `.env` na raiz, a partir do `.env.example`, com GEMINI_API_KEY e GEMINI_MODEL. O arquivo não entra no repositório nem no ZIP. Sem ele, o programa funciona em modo offline.

## Executar os testes

PowerShell

```
python -m unittest discover -s tests -v
python -m unittest tests.test_requisitos_sprint4 -v   # apenas os critérios da Sprint 4
```

## Roteiro de demonstração

| Passo | Ação | O que mostrar |
|---|---|---|
| 1 | Criar o perfil e aceitar o tour guiado | Diagnóstico completo de uma foto de exemplo |
| 2 | Opção 2 com samples/exemplo_escuro.jpg | Resumo simples e as três alternativas |
| 3 | Opção 4 com samples/exemplo_nitido.jpg | Ganho real, nota do usuário e peso atualizado |
| 4 | Opção 10: criar e processar solicitação | Arquivo de resultado em integracao/saida/ |
| 5 | Abrir data/perfis/perfil_1.json | Persistência em JSON, isolada por perfil |
| 6 | Trocar de perfil e abrir a opção 7 | O aprendizado não é compartilhado entre usuários |

*Tabela 13 — Sugestão de roteiro para a apresentação.*

SEÇÃO 13

# Limitações, referências e conclusão

## Limitações conhecidas

- Os limiares das métricas são heurísticos e precisariam de calibração com mais fotos reais.
- A pontuação depende do objetivo informado e não mede valor artístico.
- A IA pode errar; por isso sua resposta é validada, rotulada e nunca tratada como prova.
- A integração usa pastas locais compartilhadas; em produção, uma API HTTP seria mais adequada.
- O programa roda no terminal e não controla diretamente o hardware da câmera.

## Referências

- Google AI for Developers. *Image understanding*. ai.google.dev/gemini-api/docs/image-understanding
- Google AI for Developers. *Structured output*. ai.google.dev/gemini-api/docs/structured-output
- Android Developers. *Camera extensions*. developer.android.com/media/camera/camera-extensions
- Python Software Foundation. *json*, *os.replace* e *unittest*. docs.python.org/3
- OpenCV. *Image processing documentation*. docs.opencv.org

## Conclusão

A Sprint 4 entrega a versão final do JOVI AI LensLab. O programa mantém o ciclo de medir, interpretar, simular, verificar e aprender, agora com persistência em arquivos JSON, consumo da API Gemini com tratamento de falhas e integração por arquivos com o app da câmera JOVI. Cada critério da avaliação é comprovado por testes automatizados que acompanham o código entregue.